Practical SAT Solving

Lecture 6

Prof. Dr. Carsten Sinz

Hochschule Karlsruhe, University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik
Lehrgebiet Software-Entwicklung



Today

Conflict-Driven Clause Learning

CDCL

Iterative Reformulation of DPLL

```
boolean DPLL(ClauseSet S) {
  Trail := new Stack()
  while (not all variables assigned by Trail) {
    if (unitPropagation(S, Trail) = CONFLICT) {
      L = last literal not tried both True and False
      if (no such L) return UNSATISFIABLE
      pop all literals after and including L from Trail
      push {L = 0} on Trail
    } else {
      L = pick an unassigned literal
      push {L = 1} on Trail
    }
  }
  return SATISFIABLE
}
```

Reminder: Saturation Algorithm

Saturation Algorithm

- INPUT: CNF formula F
- OUTPUT: $\{SAT, UNSAT\}$

while (true) **do**
  $R = \text{resolveAll}(F)$
  if $(R \cap F \neq R)$ **then** $F = F \cup R$
  else break
if $(\bot \in F)$ **then return** $UNSAT$ **else return** SAT

Properties of the saturation algorithm:

- it is sound and complete – always terminates and answers correctly
- has exponential time and space complexity (always for Pigeons)

Search-Directed Resolution

Reminder: Resolution

- General deduction rule for clauses: $(A \vee l), (B \vee \bar{l}) \models (A \vee B)$
- Resolution Operator: $C \circ_l D$ denotes the resolvent of clauses C and D on literal l
- Can be used to extend DPLL by adding resolvents to the formula
- Special Case: Unit Resolution (central part of DPLL Search)

CDCL as Search-Directed Resolution

- CDCL can be understood as a combination of Search (like in DPLL) and Resolution (like in the Saturation Algorithm)
- At each conflict a new clause is learned
- The learned clause is obtained by resolution

Terminology

Trail

Partial variable assignment representing the path in the search tree

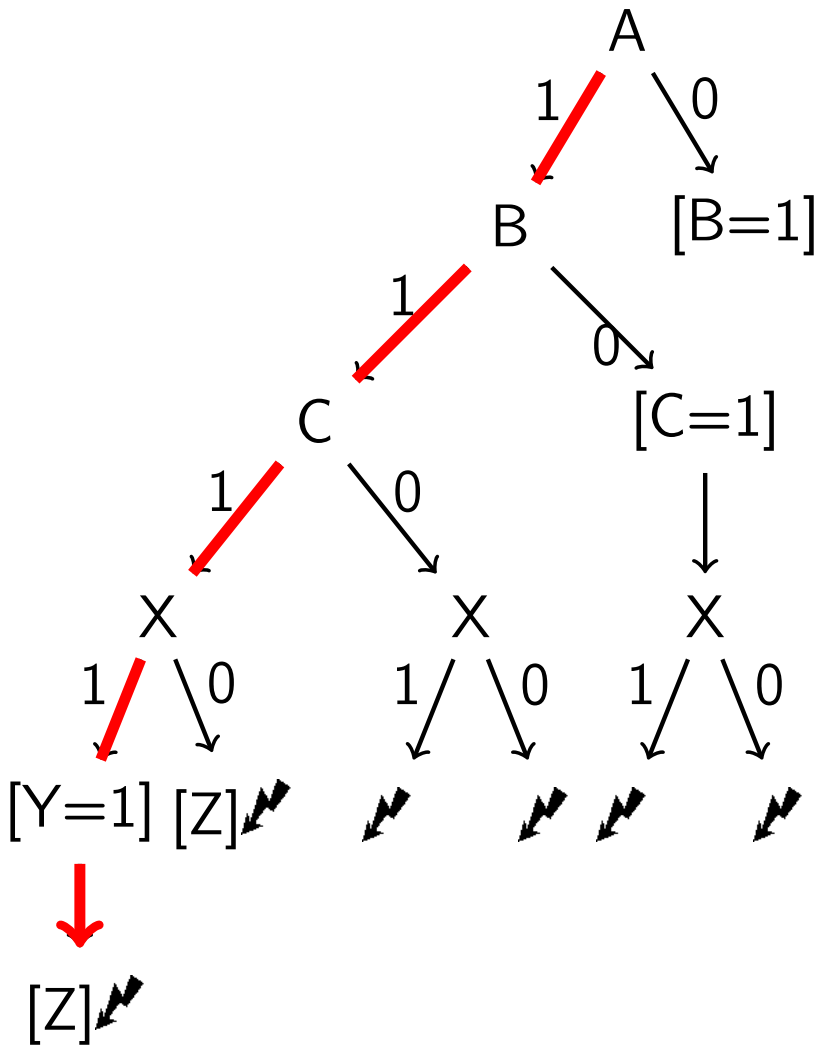
Decision Level

Number of branching literals on the current trail

Conflicting Clause

Clause with all literals assigned to 0 on the current trail

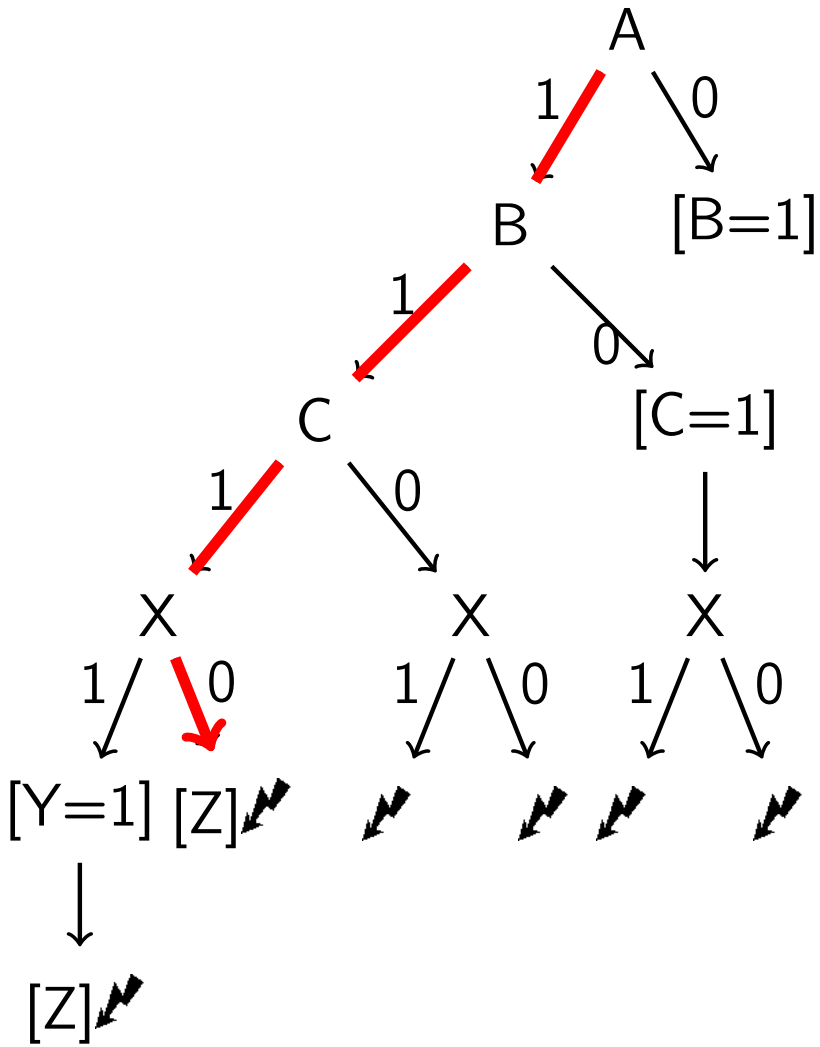
DPLL: Chronological Backtracking



$$F = \{\{A, B\},$$
$$\{B, C\},$$
$$\{\neg A, \neg X, Y\},$$
$$\{\neg A, X, Z\},$$
$$\{\neg A, \neg Y, Z\},$$
$$\{\neg A, X, \neg Z\},$$
$$\{\neg A, \neg Y, \neg Z\}\}$$

- Trail:
  A, B, C, X, Y, Z
- Conflicting Clause:
  $\{\neg A, \neg Y, \neg Z\}$

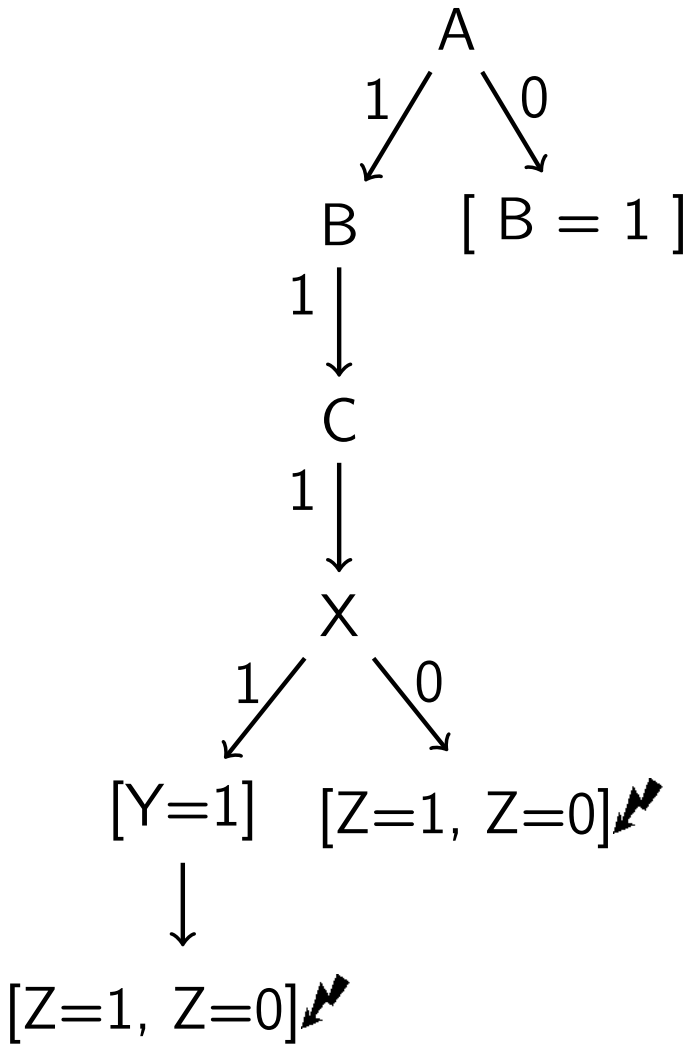
DPLL: Chronological Backtracking



$$F = \{\{A, B\},$$
$$\{B, C\},$$
$$\{\neg A, \neg X, Y\},$$
$$\{\neg A, X, Z\},$$
$$\{\neg A, \neg Y, Z\},$$
$$\{\neg A, X, \neg Z\},$$
$$\{\neg A, \neg Y, \neg Z\}\}$$

- Trail: $A, B, C, \neg X, Z$
- Conflicting Clause: $\{\neg A, X, \neg Z\}$

Non-Chronological Backtracking – Backjumping



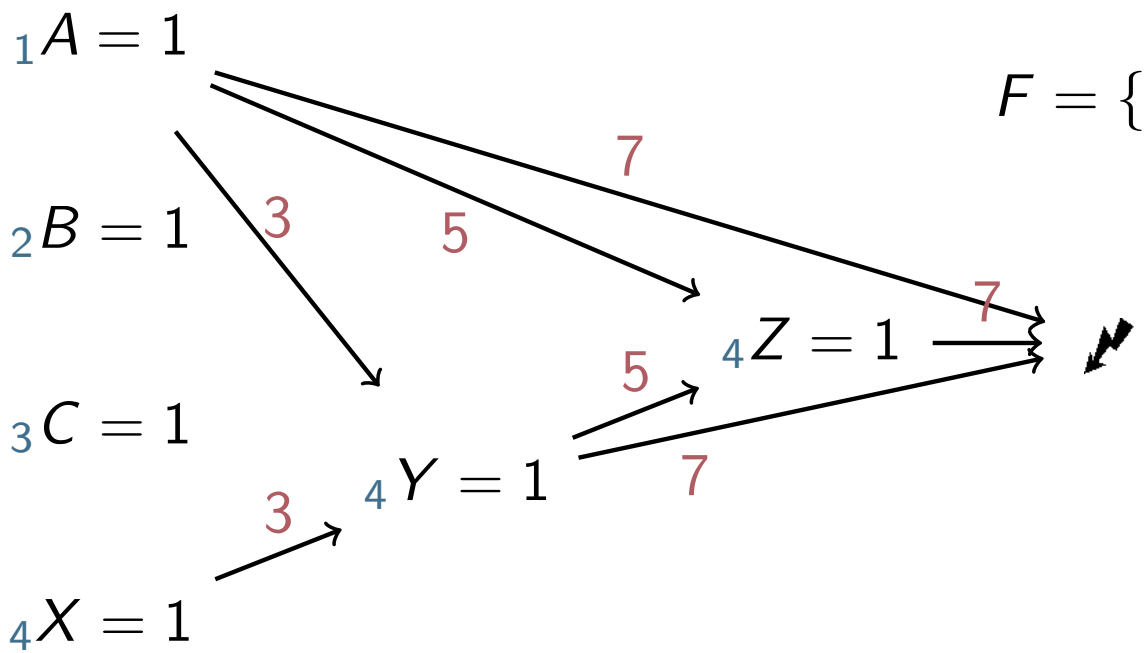
- The first two conflicting clauses $\{\neg A, \neg Y, \neg Z\}$, $\{\neg A, X, \neg Z\}$ contain only a fraction of the assignments on the trail
- Assignments to B and C obviously play no role in the present conflicting state and we could immediately backtrack to flip the assignment to A
- How to find out which assignments on the trail are relevant for the actual conflict?

Implication Graph

Given formula a F and a trail T with a conflicting clause $C \in F$, the implication graph $G = (V \cup \{\lightning\}, E)$ is defined as follows. G is a DAG, and the vertices $v \in V$ are tuples $v := [x = b, d]$ consisting of an assignment $x = b$ (given by T) and its decision level d (on T). For each clause $C = \{l_1, \ldots, l_k, u\}$ that triggered unit propagation of a literal u there are edges $e := ([l_i = 0, d_i], [u = 1, d_i])$ in E. The node with the conflicting assignment is symbolized by $\lightning$. The literals of the conflicting clause C have all edges to $\lightning$.

Example: Implication Graph

The graph shows the implication graph for the conflicting state under the trail A, B, C, X, Y, Z. The edge labels denote clauses, node labels indicate an assignment and its decision level.

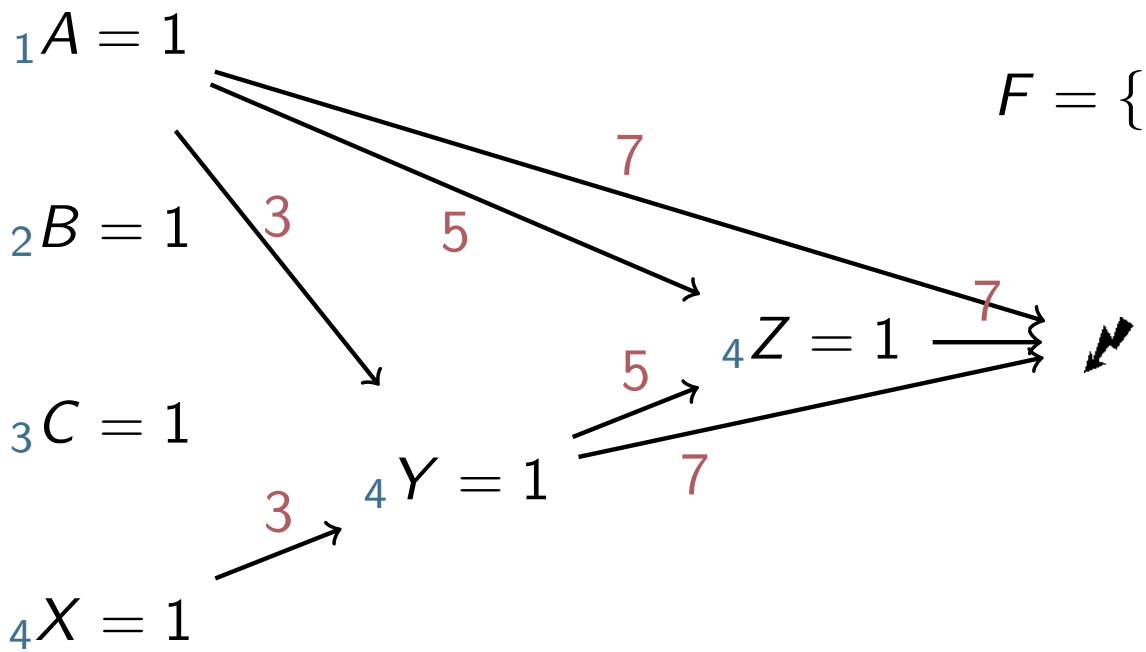


$$
\begin{aligned}
F = \{&\{A, B\}, && =: 1\\
&\{B, C\}, && =: 2\\
&\{\neg A, \neg X, Y\}, && =: 3\\
&\{\neg A, X, Z\}, && =: 4\\
&\{\neg A, \neg Y, Z\}, && =: 5\\
&\{\neg A, X, \neg Z\}, && =: 6\\
&\{\neg A, \neg Y, \neg Z\}\} && =: 7
\end{aligned}
$$

Example: Implication Graph

In the implication graph

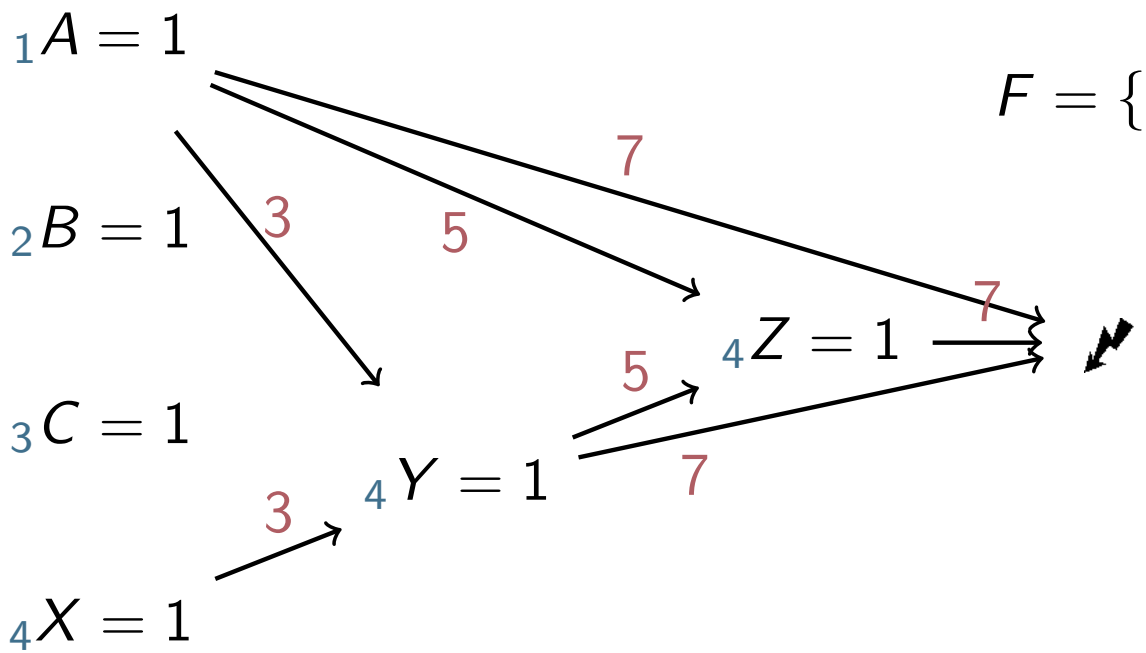
- the sink is always the conflicting assignment
- the sources are the desicion literals that take part in the conflict
- we can use it to detect the *reasons* for a conflict



$$
\begin{aligned}
F = \{&\{A, B\}, && =: 1\\
&\{B, C\}, && =: 2\\
&\{\neg A, \neg X, Y\}, && =: 3\\
&\{\neg A, X, Z\}, && =: 4\\
&\{\neg A, \neg Y, Z\}, && =: 5\\
&\{\neg A, X, \neg Z\}, && =: 6\\
&\{\neg A, \neg Y, \neg Z\}\} && =: 7
\end{aligned}
$$

Example: Conflict Analysis

In the first conflict of the previous example we can learn the clause $\{\neg A, \neg X\}$ in order to prevent the solver to pick the same partial assignment again. This can also be expressed as the following sequence of resolution steps along the edges in the implication graph: $C = (7 \circ_Z 5) \circ_Y 3$



$$
\begin{aligned}
F = \{&\{A, B\}, && =: 1\\
&\{B, C\}, && =: 2\\
&\{\neg A, \neg X, Y\}, && =: 3\\
&\{\neg A, X, Z\}, && =: 4\\
&\{\neg A, \neg Y, Z\}, && =: 5\\
&\{\neg A, X, \neg Z\}, && =: 6\\
&\{\neg A, \neg Y, \neg Z\}\} && =: 7
\end{aligned}
$$

Implementation: Implication Graph

Trail with conflicting clause $\{\neg A, \neg Y, \neg Z\}$:

↯	4	$\{\neg A, \neg Y, \neg Z\}$
Z	4	$\{\neg A, \neg Y, Z\}$
Y	4	$\{\neg A, \neg X, Y\}$
X	4	null
C	3	null
B	2	null
A	1	null

- for each assignment store pointer to reason clause on stack
- decision literals store a null pointer
- with the clause pointers and the trail we can trace all the implications back to their sources

Implementation: Conflict Analysis

Trail with conflicting clause $\{\neg A, \neg Y, \neg Z\}$:

↯	4	$\{\neg A, \neg Y, \neg Z\}$
Z	4	$\{\neg A, \neg Y, Z\}$
Y	4	$\{\neg A, \neg X, Y\}$
X	4	null
C	3	null
B	2	null
A	1	null

Trail Resolution:

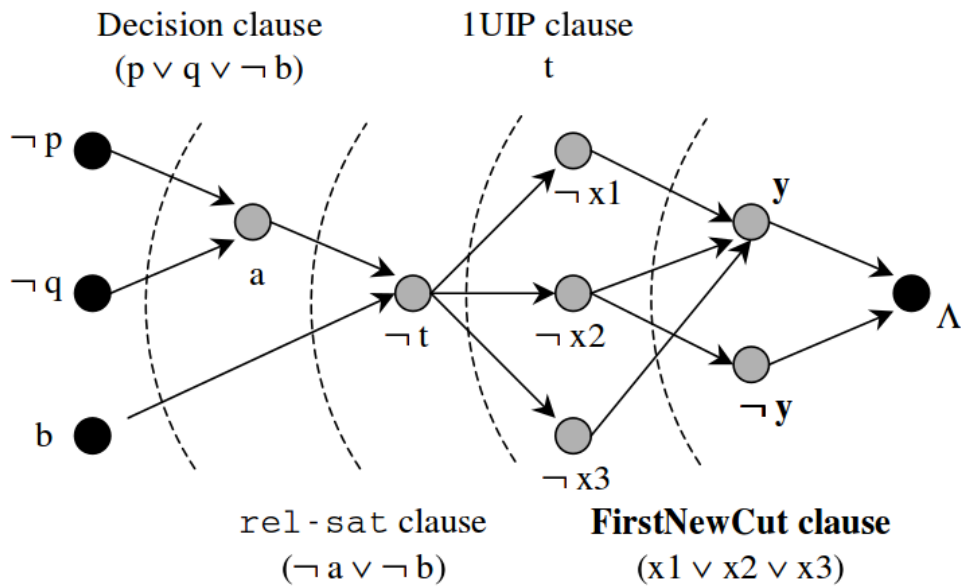
- $\{\neg A, \neg Y, \neg Z\}, \{\neg A, \neg Y, Z\} \models \{\neg A, \neg Y\}$
- $\{\neg A, \neg Y\}, \{\neg A, \neg X, Y\} \models \{\neg A, \neg X\}$
- Conflict Clause $C = \{\neg A, \neg X\}$
- Backtrack Level 1

Properties of conflict clause C:

- $F \models C$
- $F \cup \neg C \vdash_{UP} \bot$
- $D \notin F, \forall D \subseteq C$

Unique Implication Points (UIP)

In general: several possibilities to learn a clause from an implication graph. A learned clause can be resolved for every cut in the the implication graph.[1]



- UIP is a dominator in the implication graph
- A node v is a dominator, if all paths from the last decision to ↯ contain v
- FirstUIP: "first" dominator (seen from conflict side)

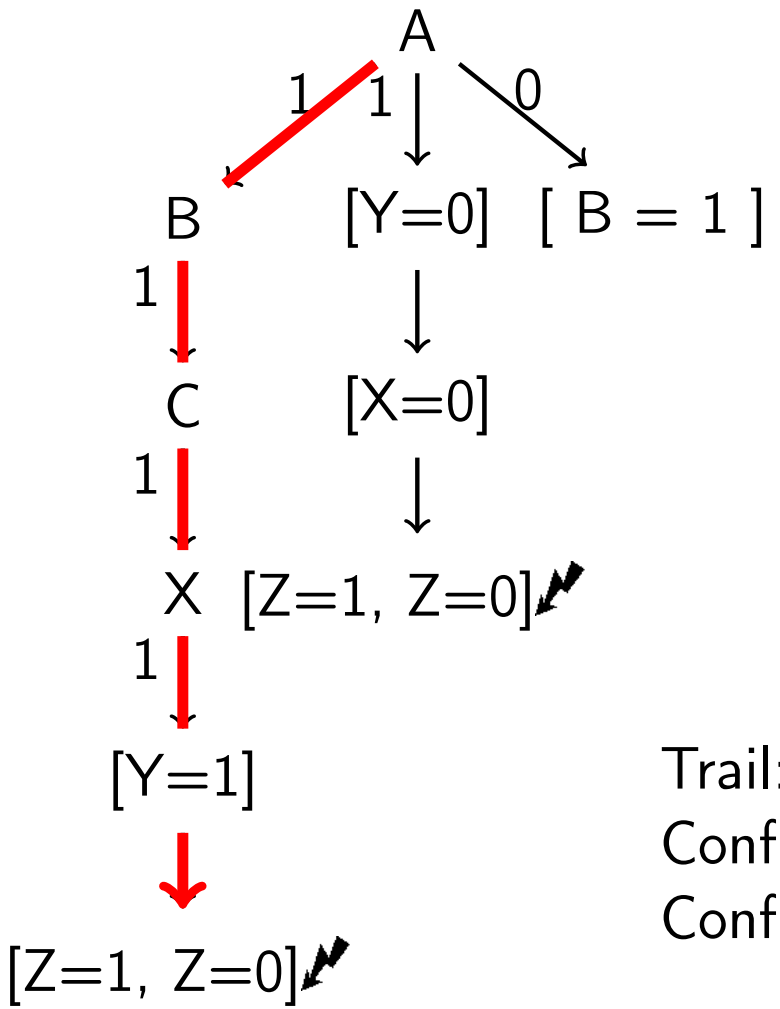
[1] Understanding the Power of Clause Learning (2003, Beame et al.)

1-UIP Learning

- FirstUIP-clause: resolve conflicting clause and reason clauses until only a single literal of the current decision level remains
- Advantage: Stopping at a UIP always leads to an asserting clause. Algorithm becomes easier: backtrack until clause becomes asserting
- Undo all decisions until that level where the learned clause becomes asserting. The assertion level is the second highest level in a conflict clause.

Backtracking with Clause Learning

In our previous example the conflict clause (1UIP) with backtracking to assertion level changes the decision tree like this:

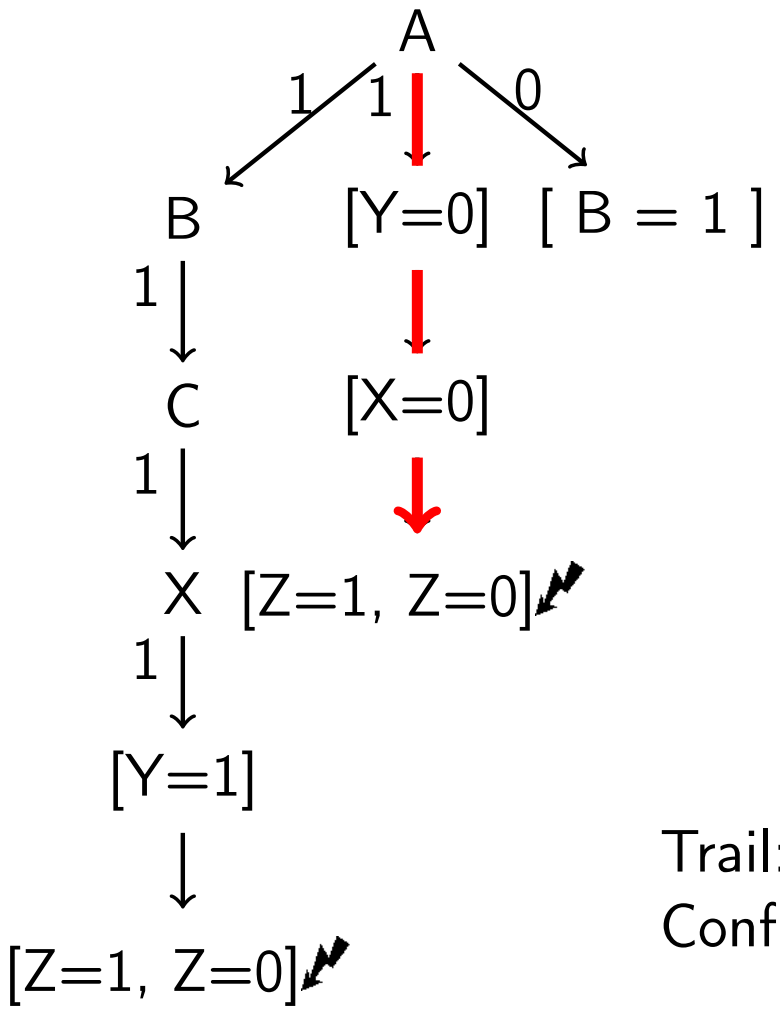


$$F = \{\{A, B\}, \{B, C\}, \{\neg A, \neg X, Y\}, \{\neg A, X, Z\}, \{\neg A, \neg Y, Z\}, \{\neg A, X, \neg Z\}, \{\neg A, \neg Y, \neg Z\}\}$$

Trail: A, B, C, X, Y, Z
Conflicting Clause: $\{\neg A, \neg Y, \neg Z\}$
Conflict Clause (1UIP): $\{\neg A, \neg Y\}$

Backtracking with Clause Learning

In our previous example the conflict clause (1UIP) with backtracking to assertion level changes the decision tree like this:



$$F = \{\{A, B\}, \{B, C\}, \{\neg A, \neg X, Y\}, \{\neg A, X, Z\}, \{\neg A, \neg Y, Z\}, \{\neg A, X, \neg Z\}, \{\neg A, \neg Y, \neg Z\} \{\neg A, \neg Y\}\}$$

Trail: $A, \neg Y, \neg X, Z$

Conflicting Clause: $\{\neg A, X, \neg Z\}$

Resolution Proof

CDCL can derive a complete resolution refutation:

- proof can serve as a certificate for validating the correctness of the SAT solver[2]
- resolution refutations based on clause learning find key practical applications (e.g. model checking)
- can help to determine minimally unsatisfiable subsets in an unsatisfiable formula

[2] `http://www.cs.utexas.edu/~marijn/drat-trim/`

Learning and Forgetting

Without restrictions the number of clauses grows exponentially.

- risk of running out of memory
- slows down unit propagation (!)
- periodic *forgetting* based on clause quality:
  - Clause Size
  - Clause Activity
  - Literal Block Distance (LBD): Number of Decision Levels
    "Predicting Learnt Clauses Quality in Modern SAT Solvers"
    (Audemard & Simon, IJCAI 2009)

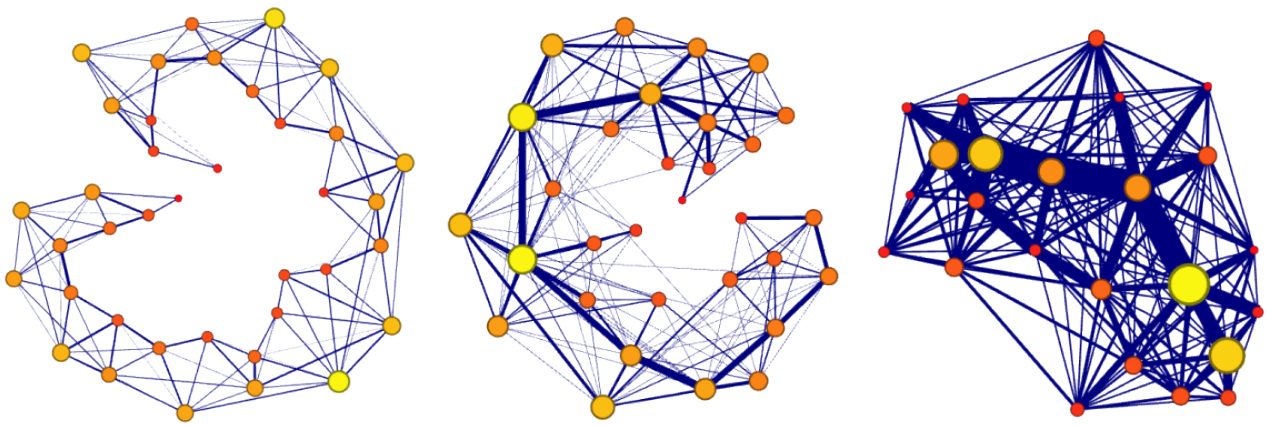
Community Structure and LBD

"Impact of Community Structure on SAT Solver Performance"

- Newsham et al., SAT 2014
- LBD correlates with number of touched communities

"Community Structure in Industrial SAT Instances"

- Ansotegui et al., AIJ 2019



Recent Successes

"Chronological Backtracking"

- Nadel & Ryvchin, SAT 2018
- Backtrack chronologically iff number of untouched decision levels is higher than a given bound
- SAT Competition 2018: `MapleLCMDistChronoBT` (best score)
- SAT Race 2019: `MapleLCMDistChronoBT` & `CaDiCal` (best scores)

"Clause Size Reduction with all-UIP Learning"

- Feng & Bacchus, SAT 2020
- 1-UIP clause has smallest LBD: continue conflict resolution as long as LBD does not increase but clause size decreases
- SAT Competition 2020: `CaDiCaL AllUIP` (won planning track)

Summary: CDCL

Algorithm 1: Conflict-driven Clause Learning (CDCL)

Input: CNF Formula F
Output: `SAT` or `UNSAT`

Data: Assignment A $\leftarrow \emptyset$
Data: Clauses C $\leftarrow F$

```
1  while A is not complete do
2      UNIT PROPAGATION
3      if A falsifies a clause in C then
4          if branching is at level 0 then
5              return UNSAT
6          else
7              (conflict-clause, backtrack-level) ←
                CONFLICT-ANALYSIS
8              backtrack to backtrack-level
9              add conflict-clause to C
10     if RESTART is triggered then
11         backtrack to level 0
12     if CLEANUP is triggered then
13         forget some of the learned clauses in C \ F
14     BRANCHING
15 return SAT
```